IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

TP1 – Exécution Java, gestion de la mémoire et débogage

Informatique – MAPD

Objectifs pédagogiques

- Écrire et exécuter un petit programme Java avec IntelliJ IDEA
- Redéfinir une méthode en Java et utiliser l'annotation `@Override` à bon escient
- Justifier un choix de conception : mutabilité/immutabilité des nombres
- Produire une documentation Java, avec Javadoc, en utilisant les étiquettes (*tags*) de base
- Utiliser les éléments fondamentaux du débogage d'un programme Java avec IntelliJ IDEA

Programmation

Exercice 1 (*Classe Rational*)

Le but de cet exercice est de programmer une classe `Rational` qui permet de décrire les nombres rationnels.

Un rationnel est le quotient de deux entiers (dont le dénominateur n'est pas nul) et qui permet de calculer des sommes, des produits, l'inverse et l'opposé.

Nous allons appliquer la méthode des petits pas : on crée d'abord une version très simple puis on écrit des tests qu'on exécute, puis on complète la classe petit à petit.

▷ **Question 1.1 :**
Écrivez une version sans exception de la classe `Rational` avec un constructeur, un point d'entrée `main` qui instancie plusieurs rationnels (dont un avec un dénominateur nul) et les affiche (avec la méthode `toString()` de `Object`).

▷ **Question 1.2 :**
Redéfinissez les méthode `toString()` et `equals()`. Exécutez à nouveau.

▷ **Question 1.3 :**
Ajoutez une méthode pour additionner deux rationnels. Ajoutez quelques appels à cette méthode dans le point d'entrée. Exécutez à nouveau.

▷ **Question 1.4 :**
Pour réaliser l'addition, vous avez du faire un choix de conception : changer la valeur du rationnel receveur (objet mutable) *ou* en retourner un nouveau (objet non mutable). Justifiez votre choix.

▷ **Question 1.5 :**
Répétez la démarche précédente pour les opérations qui permettent de multiplier, retourner l'opposé, soustraire...

Pour calculer le pgcd de deux entiers, on peut utiliser le code suivant :

```
private static int gcd(int a, int b) {
  for(int i = Math.min(a, b); i > 0; i--){
    if(a % i == 0 && b % i == 0) return i;
  }
  return 1; // never reached but makes function compilable
}
```

▷ **Question 1.6 :**
(Avancé) Faites en sorte que les rationnels retournés soient toujours irréductibles (rapport d'entiers premiers entre eux.).

Exercice 2 (*Javadoc*)

Le langage Java permet d'ajouter des commentaires qui serviront à générer la documentation (`html`) de vos programmes.

▷ **Question 2.1 :**
Ajoutez des commentaires Javadoc à la classe `Rational` puis à chacune de ses méthodes et générez la documentation.

Pour commencer, vous pouvez regarder les exemples données dans la documentation de référence de Javadoc, partie *Format of a Doc Comment*, ainsi que la documentation de l'IDE, notamment la partie *Write Javadoc comments*.

Exercice 3 (*debug*)

Nous allons observer l'exécution de votre programme à l'aide du débogueur.

▷ **Question 3.1 :**
Ajoutez un point d'arrêt à l'entrée du `main`, en cliquant sur la marge à gauche. Lancez l'exécution en mode *debug*, signalée par un *bug* vert.

▷ **Question 3.2 :**
Observez le mode de contrôle pas-à-pas du débogueur et la possibilité d'observer l'état de l'exécution. Commentez et observez les actions :
- ***Step over***
- ***Step out***
- ***Step return***
- ***Les variables locales, les arguments, this.***

Voir le tutoriel de l'IDE.

Pour aller plus loin, voir la documentation de l'IDE. Il peut notamment être utile de regarder les possibilités de raffiner les points d'arrêt (ajout d'une condition, point d'arrêt sur les champs).